

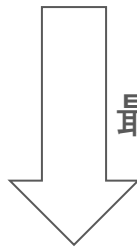
FPGA上の簡易RISC-V CPUを 用いた インベーダー風ゲームの実装

1w242304 濱谷康生

1.目的

- ・アーキテクチャAで勉強した簡易CPUを実際に動かしたい
- ・映像出力をする場合にどういう計算とリソースが必要か

⇒HDMI出力（主にデモ利用）とスプライト方式



最終目標

操作可能なゲーム(風の何か)をつくる

2. triscvの拡張

追加した機能

- シフト・論理演算：SLL / SRL / SRA, XOR
- 比較・条件分岐：SLTU, BNE, BLT / BGE, BLTU / BGEU

GitHub公開版では、教科書由来のtriscv RTLを含めず、RISC-V ISA仕様から新規に書き起こしたRV32Iコアへ置換

- 命令・データメモリ：64 words → 512 words（最終的に使うのは300語くらいになったはず）
- tas.pyも追加命令へ対応
- 命令デコーダ部分も合わせてテコ入れ

CPU(というかプログラム部分)の担当

- KEY入力，座標更新，当たり判定，敵55体の生存管理，スコア

3. 320×180フレームバッファの実装

320×180の画面RAMを用意し、デモにあったFHD映像へ縦横6倍して表示

CPUとvideo回路の接続

- CPU：32bit単位で書き込み（下位24bitがRGB）

⇒quartusの推論が勝手に削減してくれてラッキー

- video側：別の読み出しポートから一定周期でRGBを取得
- 同期RAMの1クロック遅延に合わせてDE / HSync / VSyncを遅延

座標計算と容量

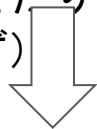
- 座標 = $y \times 320 + x = (y \ll 8(256)) + (y \ll 6(64)) + x$
- 57,600画素 × 24bit RGB = 1,382,400bit（約1.38Mbit）
5.57Mbitがde10nanoのMemBlockの値

4.当初の描画方式と問題点

①CPUで全画面(320*180)をバッファに書込

②書き込みにかかわらず読み出しは一定周期

⇒書き込み完了自体は読み出しのタイミングより早いはずなのにティアリング(Quartusでタイミングも合わせたり, 一面書込後のwaitを入れても改善せず)



ダブルバッファリングも考えたが

木村先生にご助言いただきスプライト方式へ

5. スプライト方式

画面RAMは背景として残し，出力時にゲーム物体を重ねる形に

CPU側

- KEY入力，座標更新，当たり判定
- 敵55体のalive bitとスコアをMMIOへ書き込む

合成順序

framebuffer（背景），自機sprite，敵グリッド・敵弾，自機弾，スコア表示，HDMI出力 の順番

動く物体の状態だけを更新して描画範囲のみ管理



ファミリーベーシック搭載標準スプライト
<https://yosshin4004.github.io/famibe/develop/index.html> より



ファミリーベーシック
<https://www.youtube.com/watch?v=PHqwMrAduKY>
より

6. スプライト描画とアニメーション

表示方法

- CPUが座標／有効／生存状態をMMIOへ書く
- 映像側で画素座標と形状bitを比較
- bit=1の画素だけ背景RGBを置換

敵スプライト

- 55体を5×11 gridとして描画
- 上／中／下段の3形状, 各2コマ（上だけ1体）
- 1px移動ごとに全55体を一斉に切り替える
- 試行段階では左半分3bitを保持し, 右半分を鏡面生成...みたいな感じにした
- 288bit→144bitにできた
- 最終案だと8*12にしたが, 理屈は同じ

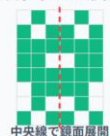
敵スプライトの構成

左右対称を利用した鏡面生成

保存する左半分



表示時の6bit形状



1行の例

保存: 011

表示: 011 | 110

3種類 × 2コマ × 8行

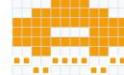
288bit → 144bit (50%削減)

実際に使用した3種類 × 2コマ

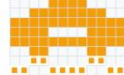
上段 squid型

配置: 1行

コマA



コマB



中段 crab型

配置: 2行

コマA



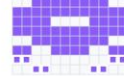
コマB



下段 octopus型

配置: 2行

コマA



コマB



白枠: コマ間で変化する画素

歩行アニメーション

敵全体の座標
enemy_base_y[0]



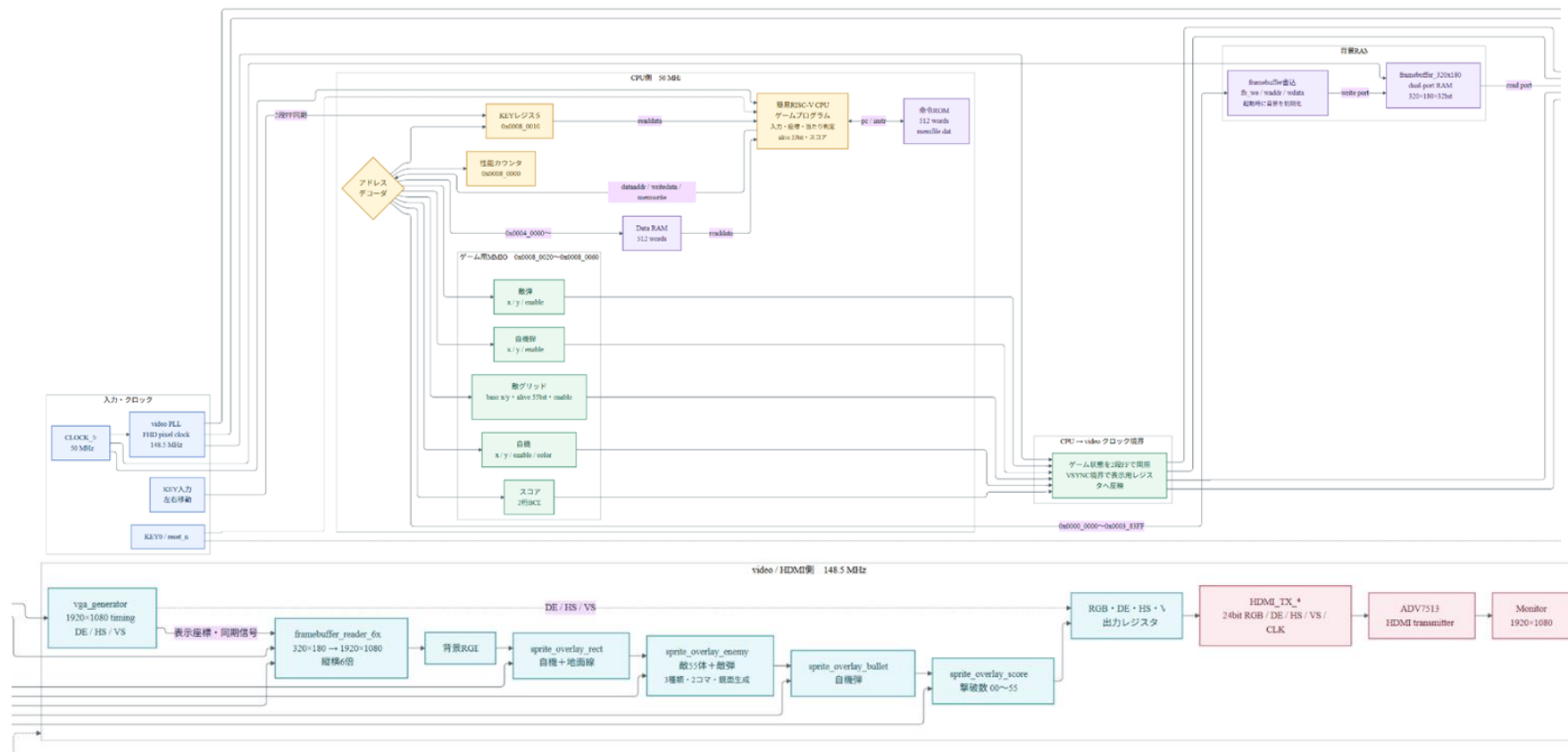
0 / 1 をコマ番号に使用



敵が1px移動するたび
55体を一斉に切替

形状データは共通
表示色は列ごとに付与

7.1全体像



7.2MMIO一覧(抜粋)

表1 性能計測・入力・自機／自機弾

アドレス	名前	R/W	有効bit	内容／初期値
0x0008_0000	PERF_START	W	値不問	計測開始・counter=0
0x0008_0004	PERF_STOP	W	値不問	計測停止・結果を保存
0x0008_0008	PERF_RESULT	R	[31:0]	計測クロック数
0x0008_000C	PERF_STATUS	R	[1],[0]	running／60Hz判定(無意味)
0x0008_0010	KEY_STATE	R	[1:0]	KEY1／KEY0 (押下=1)
0x0008_0020	SPRITE0_X	R/W	[8:0]	自機X／初期295
0x0008_0024	SPRITE0_Y	R/W	[7:0]	自機Y／初期82
0x0008_0028	SPRITE0_ENABLE	R/W	[0]	自機表示／初期0
0x0008_002C	SPRITE0_COLOR	R/W	[23:0]	RGB／初期0x00FF00
0x0008_0030	SPRITE1_X	R/W	[8:0]	自機弾X／初期0
0x0008_0034	SPRITE1_Y	R/W	[7:0]	自機弾Y／初期0

表2 自機弾・敵・スコア

アドレス	名前	R/W	有効bit	内容／初期値
0x0008_0038	SPRITE1_ENABLE	R/W	[0]	自機弾表示／初期0
0x0008_0040	ENEMY_BASE_X	R/W	[8:0]	敵grid基準X／初期20
0x0008_0044	ENEMY_BASE_Y	R/W	[7:0]	敵grid基準Y／初期18
0x0008_0048	ENEMY_ALIVE_LO	R/W	[31:0]	敵0-31生存／初期全1
0x0008_004C	ENEMY_ALIVE_HI	R/W	[22:0]	敵32-54生存／初期全1
0x0008_0050	ENEMY_ENABLE	R/W	[0]	敵表示／初期0
0x0008_0054	SCORE_BCD	R/W	[7:4],[3:0]	十の位／一の位・初期00
0x0008_0058	ENEMY_BULLET_X	R/W	[8:0]	敵弾X／初期0
0x0008_005C	ENEMY_BULLET_Y	R/W	[7:0]	敵弾Y／初期0
0x0008_0060	ENEMY_BULLET_ENABLE	R/W	[0]	敵弾表示／初期0

実機動作動画



- KEY操作による自機移動
 - 弾の自動発射，敵弾，当たり判定
 - 敵3種類の2コマ切替
 - 撃破数表示
-
- 端の列が全滅した場合，次の生存列を新しい端として判定
 - alive bitとbase座標を同時に補正し，敵の表示位置を保ったまま端判定をずらす

IoT設計とプロ研を通してFPGAを触って

- レジスタでいろいろな情報を管理して必要に応じて使う方法

⇒OS Aで学習したが、コンパイラもOSも通していないのに、verilogを書いて設定するだけで動くFPGAの自由さ！レジスタもほぼ自由

- ハード部分で全部完結させていた昔のアーケードの凄さと今簡単にプログラムを書いて軽く映せるということの楽しさ

- (デモを多分に活用したので、学べたのは僅かだが)映像出力の大変さ

- 安全性を考えないときのMMIOの楽しさ (足していくだけ、デバイスファイルなどを考えなくていい)

インベーダーゲームの歴史：https://artsandculture.google.com/story/wAVxCu_0BaX0aw?hl=ja

アタリ Sprint 2の技術仕様：<https://www.arcade-museum.com/manuals-videogames/S/Sprint2.pdf>

インベーダーゲーム製作者インタビュー：<https://igcc.jp/%E8%A5%BF%E8%A7%92%E5%8F%8B%E5%AE%8F1/>

Space Invaders Hardware (おそらく非公式解析というカリバースエンジニアリング(物理)のファンサイ

ト?)<https://www.computerarcheology.com/Arcade/SpaceInvaders/Hardware.html>